

Pandas y SQL

Tablas, llaves y el stack columnar moderno

Enrique Bolaños Abner Xocop

28 de julio de 2026

Veritas · Libertas · Jvstitia

Agenda

- 1.** Del array al DataFrame
- 2.** Explorar y seleccionar
- 3.** Agregar y combinar
- 4.** SQL
- 5.** Window functions
- 6.** DuckDB, Polars y Parquet
- 7.** Ejercicios
- 8.** Cierre

¿Qué le falta a un ndarray?

La clase pasada vimos que un array es rápido porque es **homogéneo y contiguo**. Pero los datos reales no son así:

- Una columna es texto, otra es fecha, otra es número
- Las filas y columnas tienen **nombre**, no solo posición
- Hay valores faltantes que no son cero

Pandas no reemplaza a NumPy: lo **envuelve**. Un DataFrame es un diccionario ordenado de columnas, y cada columna es (casi siempre) un array de NumPy con una etiqueta encima.

DataFrame = columnas NumPy + índice + nombres

Series y DataFrame contruidos a mano

```
1 import numpy as np
2 import pandas as pd
3
4 s = pd.Series([10, 20, 30], index=["a", "b", "c"])
5 s.values      # array([10, 20, 30])  <- NumPy adentro
6 s["b"]        # 20: acceso por etiqueta
7
8 df = pd.DataFrame({
9     "producto": ["cafe", "azucar", "harina"],
10    "precio": np.array([28.5, 9.0, 12.75]),
11    "kilos": np.array([120, 340, 260]),
12 })
13 df.dtypes     # object, float64, int64
```

Cada columna tiene su propio dtype: eso es lo que un ndarray no puede darnos.

Cargar y explorar en cinco minutos

tips: propinas de un restaurante, el dataset clásico de seaborn.

```
1 URL = ("https://raw.githubusercontent.com/"
2        "mwaskom/seaborn-data/master/tips.csv")
3 tips = pd.read_csv(URL)
4 tips.shape           # (244, 7)
5 tips.head()         # primeras 5 filas
6 tips.info()          # dtypes, nulos, memoria
7 tips.describe()      # estadísticos numéricos
8 tips["day"].value_counts()
9 tips.isna().sum()    # nulos por columna
```

MÁS ALLÁ DEL CÓDIGO

Nunca modelen una tabla que no exploraron: estos comandos dicen si se leyó bien y dónde faltan datos.

Seleccionar: loc, iloc y máscaras

```
1 tips["tip"] # una Series
2 tips[["tip", "total_bill"]] # un DataFrame
3 tips.loc[0:4, ["day", "tip"]] # por ETIQUETA
4 tips.iloc[0:5, 0:2] # por POSICION
5 tips[tips["total_bill"] > 30] # mascara
6 tips[(tips["day"] == "Sun") &
7      (tips["size"] >= 4)]
8 tips["pct"] = tips["tip"] / tips["total_bill"]
```

loc habla el idioma de las etiquetas, iloc el de los enteros: mezclarlos es la causa número uno de bugs en pandas.

Las máscaras son las mismas de NumPy, con nombres

groupby: split, apply, combine

Casi toda pregunta de negocio es “¿cuánto, por cada categoría?”: **partir** la tabla por una columna, **calcular** dentro de cada pedazo, **juntar** los resultados. Una fila por grupo.

```
1 tips.groupby("day")["tip"].mean()
2 tips.groupby(["day", "time"])["total_bill"].sum()
3 tips.groupby("day").agg(
4     propina_medio=("tip", "mean"),
5     cuenta_max=("total_bill", "max"),
6     n=("tip", "size"),
7 )
8 tips.groupby("day")["pct"].transform("mean")
```

agg resume (una fila por grupo); transform pega el valor del grupo a **cada fila original**. Esa diferencia reaparece en window functions.

merge: pegar tablas por una llave

Los datos reales viven repartidos. La **llave** es la columna que enlaza dos tablas.

```
1 ventas = pd.DataFrame({
2     "sucursal": [1, 2, 1, 3],
3     "monto": [250, 480, 120, 310]})
4 sucursales = pd.DataFrame({
5     "sucursal": [1, 2, 4],
6     "ciudad": ["Guatemala", "Xela", "Peten"]})
7
8 pd.merge(ventas, sucursales, on="sucursal",
9         how="inner")    # solo llaves en AMBAS
10 pd.merge(ventas, sucursales, on="sucursal",
11         how="left")    # todas las de ventas
```

Con `how="left"`, la sucursal 3 aparece con ciudad nula: eso es información, no un error.

Por qué SQL sigue siendo la lingua franca

SQL tiene más de 50 años y no se va a ir. Razones:

- Los datos de una empresa **viven en bases de datos**, no en CSV
- Es **declarativo**: describimos el resultado, no el algoritmo
- El motor decide cómo ejecutarlo, y suele decidir mejor que nosotros
- Lo entienden analistas, ingenieros y herramientas de BI

Lo importante hoy: merge y JOIN son la misma operación, groupby y GROUP BY son la misma idea. Cambia la sintaxis, no el concepto.

MÁS ALLÁ DEL CÓDIGO

En una entrevista de trabajo de datos les van a preguntar SQL casi con certeza. Vale la pena escribirlo hasta que salga sin pensar.

SQL esencial en una sola query

```
1 SELECT day,  
2         AVG(tip) AS propina_media,  
3         COUNT(*) AS n  
4 FROM tips  
5 WHERE total_bill > 10  
6 GROUP BY day  
7 HAVING COUNT(*) > 20  
8 ORDER BY propina_media DESC  
9 LIMIT 3;
```

El motor no la lee de arriba hacia abajo: primero FROM, luego WHERE (filtra **filas**), luego GROUP BY, luego HAVING (filtra **grupos**), y hasta el final SELECT y ORDER BY. Por eso no se puede usar un alias del SELECT dentro del WHERE.

JOIN: el merge de SQL

```
1 SELECT v.sucursal, s.ciudad, SUM(v.monto) AS total
2 FROM ventas AS v
3 INNER JOIN sucursales AS s
4         ON v.sucursal = s.sucursal
5 GROUP BY v.sucursal, s.ciudad
6 ORDER BY total DESC;
```

- **INNER JOIN**: solo las llaves presentes en ambas tablas
- **LEFT JOIN**: todas las de la izquierda, con nulos donde falte

Si el resultado tiene más filas de las que esperaban, la llave estaba duplicada en alguna tabla. Siempre verifiquen el shape después de un join.

Window functions: resumir sin colapsar

Un GROUP BY devuelve una fila por grupo. Una **window function** calcula sobre el grupo pero **conserva todas las filas**: es el transform de pandas, en SQL.

```
1 SELECT day, total_bill, tip,  
2         RANK() OVER (PARTITION BY day  
3                       ORDER BY total_bill DESC) AS pos,  
4         AVG(tip) OVER (PARTITION BY day) AS media_dia  
5 FROM tips;
```

PARTITION BY define el grupo, ORDER BY define el orden dentro de la ventana.

Rankings y acumulados

```
1 SELECT day, size, total_bill,  
2         SUM(total_bill) OVER (  
3             PARTITION BY day  
4             ORDER BY size) AS acumulado,  
5         LAG(total_bill) OVER (  
6             PARTITION BY day  
7             ORDER BY size) AS anterior  
8 FROM tips;
```

- ROW_NUMBER, RANK, DENSE_RANK: posiciones
- SUM/AVG ... OVER (ORDER BY ...): acumulados y medias móviles
- LAG y LEAD: la fila anterior y la siguiente

Series de tiempo, cohortes, “top 3 por categoría”: todo eso son ventanas.

DuckDB: SQL sobre sus DataFrames

DuckDB es una base de datos analítica **sin servidor**: se instala con pip y consulta lo que ya tienen en memoria.

```
1 import duckdb
2
3 duckdb.sql("""
4     SELECT day, AVG(tip) AS media, COUNT(*) AS n
5     FROM tips
6     GROUP BY day
7     ORDER BY media DESC
8 """).df()
```

tips es la variable de Python: DuckDB la ve directamente. El .df() final devuelve un DataFrame de pandas, así que pueden encadenar.

Parquet y Polars: el stack columnar

CSV es texto plano: sin tipos, sin compresión, se lee entero siempre. **Parquet** es columnar, tipado y comprimido.

```
1 tips.to_parquet("tips.parquet")
2 duckdb.sql("""SELECT day, SUM(tip)
3               FROM 'tips.parquet'
4               GROUP BY day""").df()
5 import polars as pl
6 (pl.scan_parquet("tips.parquet")
7  .filter(pl.col("total_bill") > 20)
8  .group_by("day").agg(pl.col("tip").mean())
9  .collect())
```

Leer 2 columnas de 200 no cuesta lo mismo que leer las 200. Y `scan_parquet` no lee nada hasta `collect()`: eso es **lazy evaluation**, y permite archivos más grandes que la RAM.

¿Cuándo uso cada uno?

Herramienta	Cuándo
pandas	Exploración, datos que caben en RAM, todo el ecosistema de gráficas y scikit-learn
SQL	Los datos ya están en una base; consultas que otros deben leer y mantener
DuckDB	Quieren SQL sin base de datos: archivos locales, Parquet, agregaciones pesadas
Polars	Archivos grandes, pipelines repetidos, necesitan velocidad y lazy

En este curso usamos pandas como base y DuckDB cuando SQL sea más claro. No son rivales: conviven en el mismo notebook.

Ejercicio 1: propinas por día y turno

```
1 # a) porcentaje medio de propina por día
2 # b) por día Y turno (day, time), en una tabla
3 # c) el día con el mayor porcentaje medio
4 # d) número de mesas por día
5
6 resumen = ... # ¿Que va aquí? (usen agg)
7
8 # Verificación: resumen debe tener 4 filas
```

Ejercicio 2: un merge con llave imperfecta

```
1 metas = pd.DataFrame({
2     "day": ["Thur", "Fri", "Sat"],
3     "meta_propina": [0.16, 0.16, 0.18]})
4
5 # a) peguen metas a tips con how="left"
6 # b) ¿que dia quedo sin meta? ¿por que?
7 # c) columna cumple = pct >= meta_propina
8 # d) % de mesas que cumplen, por dia
9
10 # ¿Que va aqui?
11 # Verificacion: comparen shape antes y despues
```

Ejercicio 3: la misma pregunta, dos veces

Pregunta: *las 3 cuentas más altas de cada día, con su posición.*

```
1 # Version A: pandas
2 # Pista: sort_values + groupby(...).head(3)
3 top_pandas = ...    # ¿Que va aqui?
4
5 # Version B: DuckDB con window function
6 top_sql = duckdb.sql("""
7     ... /* ¿Que va aqui? RANK OVER PARTITION */
8     """).df()
9
10 # Verificacion: ambos deben dar 4*3 = 12 filas
```

Escribir la misma pregunta en los dos lenguajes es la mejor forma de aprender ambos.

Lo esencial de hoy

- Un DataFrame es **columnas NumPy con etiquetas** y tipos mixtos
- loc por etiqueta, iloc por posición; máscaras igual que en NumPy
- groupby es **split, apply, combine**; agg resume, transform no
- merge es JOIN: siempre revisen el shape después
- Las **window functions** resumen sin colapsar filas
- DuckDB da SQL sin servidor; Parquet y Polars son el stack columnar

Próxima clase (jueves 30): Adquisición de Datos

APIs, web scraping y datos sucios del mundo real

Se asigna la **HDT 1** (entrega: martes 4 de agosto)